Universidad Don Bosco

Materia:

Lenguajes Interpretados en el Cliente LIC941 G01T (Virtual)

Actividad:

Entrega Proyecto Fase 1

Integrantes:

N°	Grupo #5	Carné
1	Caceres Guzman, Vanessa Ivette	CG231059
2	Guardado Mendoza, Yeimy Tatiana	GM252740
3	Hayzel Evelyn, González Gutiérrez	GG240801

Docente:

Ing. Oscar Ariel Pérez Torres

Fecha de entrega:

12/04/2026

Contenido

Introducción 4
Objetivos 5
General: 5
Específicos: 5
¿Cómo ejecutar el proyecto localmente? 6
Estructura del proyecto 7
PokemonBank/ 7
js/main.js 7
css/styles.css 7
images/ 7
index.html 7
acciones.html 7
historial.html 8
grafico.html 8
Documentación 8
js/main.js 8
css/styles.css 21
images/ 33
AshKetchumPFP.jpg 33
PKBC.png 33
PKBL.png 33
index.hmtl 33
acciones.html 35
historial.html 40
grafico.html 42
Desarrollo del análisis de proyecto 44
Pantallas 44
Login 44
Acciones 45
Depositar 46
Retirar 47
Consultar saldo 48
Ver historial 49
Ver gráfico 50

Flujograma 51
Link: 51
Imagen: 51
Investigación de herramientas 52
SweetAlert 52
Ejemplos de uso en el proyecto: 52
Chart.js 54
Ejemplos de uso en el proyecto: 54
Versionamiento del código 55
Primer commit 55
main.js 56
styles.css 56
index.html 56
acciones.html 57
historial.html 57
grafico.html 57
Último commit 58
Conclusiones 58

Introducción

En un entorno donde la digitalización define nuestra interacción con los servicios financieros, nace Pokémon Bank: una aplicación web de vanguardia diseñada para simular la experiencia completa de un cajero automático (ATM). Este proyecto trasciende la simple gestión de transacciones, enfocándose en la creación de una interfaz robusta, interactiva y de alta eficiencia.

Desarrollada con el stack fundamental de HTML, CSS y JavaScript, la plataforma permite al usuario ejecutar operaciones críticas desde depósitos y retiros hasta pagos de servicios y análisis gráfico de movimientos bajo un entorno seguro de autenticación por PIN.

Pokémon Bank no es solo un simulador bancario; es la materialización de un diseño responsivo y dinámico que demuestra cómo las herramientas de frontend pueden transformar procesos complejos en una experiencia digital fluida, amigable y profesional.

Objetivos

General:

Desarrollar una aplicación web interactiva que simule el funcionamiento de un cajero automático (ATM), utilizando tecnologías frontend como HTML, CSS y JavaScript, con el fin de ofrecer una experiencia digital eficiente, segura y centrada en el usuario.

Específicos:

1. Implementar un sistema de autenticación mediante PIN que garantice el acceso seguro a la aplicación.

2. Diseñar una interfaz gráfica responsiva, intuitiva y amigable que facilite la interacción del usuario con las funcionalidades del sistema.

3. Desarrollar funcionalidades esenciales de un cajero automático, como depósitos, retiros, pagos de servicios y consulta de movimientos.

4. Integrar visualizaciones gráficas que permitan al usuario analizar sus transacciones de manera clara y comprensible.

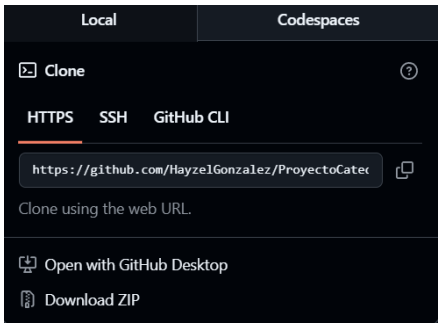
5. Aplicar buenas prácticas de desarrollo frontend para asegurar un código estructurado, mantenible y eficiente.

6. Simular procesos bancarios reales dentro de un entorno digital para fortalecer la comprensión de sistemas interactivos.

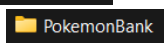
¿Cómo ejecutar el proyecto localmente?

1. Acceder a este link: https://github.com/HayzelGonzalez/ProyectoCatedra-LICG01TV--Grupo-5.git
2. Presionar este botón:

3. Luego de presionarlo, aparecerá este desplegable:

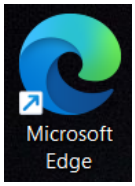


4. Presionar donde dice “Download ZIP”.
5. Luego de descargarlo, descomprimir el archivo:

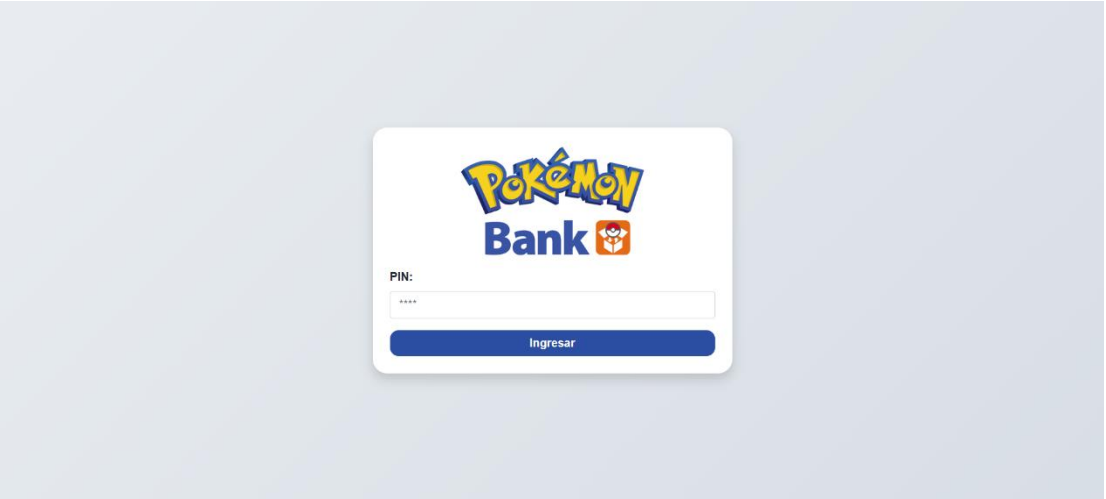
6. Posteriormente, abrir la carpeta y encontrarán una carpeta llamada: 

7. Abrir la carpeta “PokemonBank” y encontrar un archivo HTML llamado “index.html”:
8. Darle clic derecho al archivo “index.html” y aparecerá esto:
9. Presionar “Abrir con” y aparecerá esto:
10. Seleccionar un navegador para ejecutar “index.html”, preferiblemente:

o



11. Finalmente, luego de haber seleccionado el navegador, tiene que aparecer esto:



Estructura del proyecto

PokemonBank/

Contiene todo el sistema web del cajero automático simulado. Dentro de esta carpeta están las páginas, estilos, lógica y recursos visuales del proyecto.

js/main.js

Es el archivo que contiene la **lógica principal del sistema**. Controla el inicio de sesión, el cierre de sesión, los depósitos, retiros, consulta de saldo, pago de servicios, historial, gráfico y el manejo de datos en ***localStorage***.

css/styles.css

Archivo que contiene todos los **estilos visuales** del proyecto. Aquí se define la apariencia del login, la pantalla principal, los botones, los modales, el historial y el gráfico.

images/

Carpeta donde se guardan los **recursos gráficos del proyecto**, como el logo del banco y la imagen de perfil del usuario. Estos archivos se usan en las distintas interfaces del sistema.

index.html

Es la **pantalla de inicio de sesión**. Aquí el usuario ingresa su PIN para poder entrar al sistema. Desde esta página se carga la lógica principal del proyecto y se conecta con los estilos.

acciones.html

Es la **pantalla principal del cajero**. Muestra el nombre y número de cuenta del usuario, y permite acceder a las opciones de:

- Depositar
- Retirar.
- Consultar saldo.
- Pagar servicios.
- Ver historial.
- Ver gráfico.
- Cerrar sesión.

historial.html

Es la página donde se muestra el **historial de transacciones** realizadas por el usuario, como depósitos, retiros y pagos de servicios.

grafico.html

Es la página donde se visualiza el **gráfico de transacciones**. Presenta de forma gráfica los movimientos realizados por el usuario dentro del sistema.

Documentación

js/main.js

```
// ==========================================================
// USUARIO BASE
// ==========================================================
// Aquí se crea el objeto principal del usuario.
// Este objeto sirve como plantilla inicial para la app.
// Cuando alguien inicia sesión por primera vez, estos datos
// se guardan en localStorage para simular una cuenta bancaria.
const usuarioBase = {
  // Nombre del usuario que se mostrará en la pantalla principal.
  nombre: "Ash Ketchum",

  // Número de cuenta que también se mostrará en la interfaz.
  cuenta: "0987654321",

  // PIN correcto que permitirá iniciar sesión.
  pin: "1234",

  // Saldo inicial con el que empieza la cuenta.
  saldo: 500,

  // Arreglo vacío donde se guardarán las transacciones.
  historial: []
};

// ==========================================================
// FUNCIÓN: INICIAR SESIÓN
// ==========================================================
// Esta función se ejecuta cuando el usuario presiona
// el botón "Ingresar" en el login.
```

```
function iniciarSesion() {
  // Busca en el HTML el input que tiene el id "pin".
  const pinInput = document.getElementById("pin");

  // Si el input no existe, la función se detiene.
  // Esto evita errores si se ejecuta en otra página.
  if (!pinInput) return;

  // Toma el valor escrito en el input
  // y elimina espacios al inicio o al final.
  const pinIngresado = pinInput.value.trim();

  // Compara el PIN ingresado con el PIN del usuario base.
  if (pinIngresado === usuarioBase.pin) {
    // Busca si ya existe un usuario guardado en localStorage.
    const usuarioGuardado = localStorage.getItem("usuarioPokemonBank");

    // Si todavía no hay un usuario guardado,
    // se guarda por primera vez el usuarioBase.
    if (!usuarioGuardado) {
      localStorage.setItem("usuarioPokemonBank", JSON.stringify(usuarioBase));
    }

    // Muestra un mensaje de acceso correcto.
    swal({
      title: "Bienvenido",
      text: "Acceso concedido a Pokémon Bank",
      icon: "success",
      button: "Continuar"
    }).then(() => {
      // Después de cerrar la alerta,
      // redirige al usuario a la página principal.
      window.location.href = "acciones.html";
    });
  } else {
    // Si el PIN es incorrecto, muestra un error.
    swal({
      title: "PIN incorrecto",
      text: "Verifica tus datos e intenta nuevamente.",
      icon: "error",
      button: "Aceptar"
    });
  }
}

// ==========================================================
// FUNCIÓN: OBTENER USUARIO
// ==========================================================
```

```
// Esta función lee el usuario guardado en localStorage.
// localStorage guarda texto, por eso se usa JSON.parse
// para convertirlo nuevamente en objeto.
function obtenerUsuario() {
  return JSON.parse(localStorage.getItem("usuarioPokemonBank"));
}

// ==========================================================
// FUNCIÓN: GUARDAR USUARIO
// ==========================================================
// Esta función recibe un objeto actualizado
// y lo guarda otra vez en localStorage.
function guardarUsuario(usuarioActualizado) {
  localStorage.setItem("usuarioPokemonBank",
JSON.stringify(usuarioActualizado));
}

// ==========================================================
// FUNCIÓN: CERRAR SESIÓN
// ==========================================================
// Esta función se ejecuta cuando el usuario presiona
// el botón "Cerrar sesión".
function cerrarSesion() {
  // Si algún botón o elemento quedó seleccionado,
  // se le quita el foco para evitar efectos visuales raros.
  if (document.activeElement) {
    document.activeElement.blur();
  }

  // Muestra una alerta de confirmación.
  swal({
    title: "¿Está seguro de cerrar sesión?",
    text: "Se cerrará la sesión actual de Pokémon Bank.",
    icon: "warning",
    buttons: {
      confirm: {
        text: "Aceptar",
        value: true,
        visible: true
      },
      cancel: {
        text: "Cancelar",
        value: false,
        visible: true
      }
    },
    // dangerMode hace que la alerta tenga aspecto más delicado.
    dangerMode: true
```

```
  }).then((confirmado) => {
    // Si el usuario acepta cerrar sesión...
    if (confirmado) {
      // ...se elimina la sesión guardada.
      localStorage.removeItem("usuarioPokemonBank");

      // Luego regresa al login.
      window.location.href = "index.html";
    }
  });
}

// ==========================================================
// FUNCIÓN: PROCESAR DEPÓSITO
// ==========================================================
// Esta función se usa cuando el usuario confirma
// un depósito desde el modal.
function procesarDeposito() {
  // Obtiene el usuario guardado.
  const usuario = obtenerUsuario();

  // Si por alguna razón no existe usuario, detiene la función.
  if (!usuario) return;

  // Busca el valor del input del depósito
  // y lo convierte a número decimal.
  const monto = parseFloat(document.getElementById("montoDeposito").value);

  // Valida que el monto sea un número válido
  // y que sea mayor que cero.
  if (isNaN(monto) || monto <= 0) {
    swal("Monto inválido", "Ingresa un monto válido para realizar el
depósito.", "warning");
    return;
  }

  // Pide confirmación antes de ejecutar la operación.
  swal({
    title: "¿Está seguro de querer realizar esta operación?",
    text: 'Una vez presionado "Aceptar" no podrá revertir esta operación.',
    icon: "warning",
    buttons: {
      confirm: {
        text: "Aceptar",
        value: true,
        visible: true
      },
      cancel: {
```

```
        text: "Cancelar",
        value: false,
        visible: true
      }
    },
    dangerMode: true
  }).then((confirmado) => {
    // Si el usuario confirma...
    if (confirmado) {
      // ...el saldo aumenta.
      usuario.saldo += monto;

      // Luego se registra la transacción en el historial.
      usuario.historial.push({
        tipo: "Depósito",
        monto: monto
      });

      // Se guarda el usuario actualizado.
      guardarUsuario(usuario);

      // Se limpia el formulario del modal.
      document.getElementById("formDepositar").reset();

      // Se cierra el modal.
      $("#modalDepositar").modal("hide");

      // Muestra un mensaje de éxito.
      swal("Depósito realizado", "Se depositó $" + monto.toFixed(2) + "
correctamente.", "success");
    }
  });
}

// ==========================================================
// FUNCIÓN: PROCESAR RETIRO
// ==========================================================
// Esta función se usa para retirar dinero del saldo.
function procesarRetiro() {
  // Obtiene el usuario actual.
  const usuario = obtenerUsuario();

  // Si no existe usuario, se detiene.
  if (!usuario) return;

  // Obtiene el monto escrito y lo convierte a decimal.
  const monto = parseFloat(document.getElementById("montoRetiro").value);
```

```
  // Valida que el monto sea correcto.
  if (isNaN(monto) || monto <= 0) {
    swal("Monto inválido", "Ingresa un monto válido para realizar el retiro.",
"warning");
    return;
  }

  // Verifica que haya suficiente saldo.
  if (monto > usuario.saldo) {
    swal("Fondos insuficientes", "No tienes saldo suficiente para realizar
este retiro.", "error");
    return;
  }

  // Muestra confirmación antes de retirar.
  swal({
    title: "¿Está seguro de querer realizar esta operación?",
    text: 'Una vez presionado "Aceptar" no podrá revertir esta operación.',
    icon: "warning",
    buttons: {
      confirm: {
        text: "Aceptar",
        value: true,
        visible: true
      },
      cancel: {
        text: "Cancelar",
        value: false,
        visible: true
      }
    },
    dangerMode: true
  }).then((confirmado) => {
    // Si confirma...
    if (confirmado) {
      // ...se descuenta el monto del saldo.
      usuario.saldo -= monto;

      // Se guarda la transacción en historial.
      usuario.historial.push({
        tipo: "Retiro",
        monto: monto
      });

      // Se actualiza localStorage.
      guardarUsuario(usuario);

      // Se limpia el formulario del modal.
```

```
      document.getElementById("formRetirar").reset();

      // Se cierra el modal.
      $("#modalRetirar").modal("hide");

      // Se informa que el retiro fue exitoso.
      swal("Retiro realizado", "Se retiró $" + monto.toFixed(2) + "
correctamente.", "success");
    }
  });
}

// ==========================================================
// FUNCIÓN: CONSULTAR SALDO
// ==========================================================
// Esta función solo muestra el saldo actual en una alerta.
function consultarSaldo() {
  // Obtiene el usuario guardado.
  const usuario = obtenerUsuario();

  // Si no existe usuario, se detiene.
  if (!usuario) return;

  // Muestra el saldo con dos decimales.
  swal("Saldo actual", "$" + usuario.saldo.toFixed(2), "info");
}

// ==========================================================
// FUNCIÓN: PROCESAR PAGO DE SERVICIO
// ==========================================================
// Esta función descuenta dinero por el pago de un servicio
// y guarda esa operación en el historial.
function procesarPagoServicio() {
  // Obtiene el usuario guardado.
  const usuario = obtenerUsuario();

  // Si no hay usuario, detiene la función.
  if (!usuario) return;

  // Obtiene el servicio elegido en el select.
  const servicio = document.getElementById("tipoServicio").value;

  // Obtiene el monto escrito y lo convierte a decimal.
  const monto = parseFloat(document.getElementById("montoServicio").value);

  // Valida que haya un servicio seleccionado
  // y que el monto sea correcto.
  if (!servicio || isNaN(monto) || monto <= 0) {
```

```
    swal("Datos inválidos", "Selecciona un servicio e ingresa un monto
válido.", "warning");
    return;
  }

  // Verifica si hay saldo suficiente.
  if (monto > usuario.saldo) {
    swal("Fondos insuficientes", "No tienes saldo suficiente para pagar este
servicio.", "error");
    return;
  }

  // Muestra confirmación antes del pago.
  swal({
    title: "¿Está seguro de querer realizar esta operación?",
    text: 'Una vez presionado "Aceptar" no podrá revertir esta operación.',
    icon: "warning",
    buttons: {
      confirm: {
        text: "Aceptar",
        value: true,
        visible: true
      },
      cancel: {
        text: "Cancelar",
        value: false,
        visible: true
      }
    },
    dangerMode: true
  }).then((confirmado) => {
    // Si el usuario confirma...
    if (confirmado) {
      // ...se resta el monto al saldo.
      usuario.saldo -= monto;

      // Luego se agrega la transacción al historial.
      usuario.historial.push({
        tipo: "Pago de servicio - " + servicio,
        monto: monto
      });

      // Se guarda el usuario actualizado.
      guardarUsuario(usuario);

      // Se reinicia el formulario del modal.
      document.getElementById("formPagoServicio").reset();
```

```
      // Se cierra el modal.
      $("#modalPagoServicio").modal("hide");

      // Muestra mensaje de éxito.
      swal("Pago exitoso", "Se pagó " + servicio + " por $" +
monto.toFixed(2), "success");
    }
  });
}

// ==========================================================
// FUNCIÓN: CARGAR DATOS DEL USUARIO EN PANTALLA
// ==========================================================
// Esta función coloca el nombre y la cuenta
// en la pantalla principal de acciones.
function cargarDatosUsuario() {
  // Obtiene el usuario guardado.
  const usuario = obtenerUsuario();

  // Busca el elemento del nombre.
  const nombreUsuario = document.getElementById("nombreUsuario");

  // Busca el elemento del número de cuenta.
  const numeroCuenta = document.getElementById("numeroCuenta");

  // Si existe la zona donde deberían mostrarse datos
  // pero no existe usuario, regresa al login.
  if ((nombreUsuario || numeroCuenta) && !usuario) {
    window.location.href = "index.html";
    return;
  }

  // Si existe el span del nombre y sí hay usuario,
  // coloca el nombre.
  if (nombreUsuario && usuario) {
    nombreUsuario.textContent = usuario.nombre;
  }

  // Si existe el span de cuenta y sí hay usuario,
  // coloca el número de cuenta.
  if (numeroCuenta && usuario) {
    numeroCuenta.textContent = usuario.cuenta;
  }
}

// ==========================================================
// FUNCIÓN: MOSTRAR HISTORIAL
// ==========================================================
```

```
// Esta función llena la tabla de historial.
function mostrarHistorial() {
  // Busca el body de la tabla.
  const tablaHistorial = document.getElementById("tablaHistorial");

  // Busca el mensaje que indica que no hay movimientos.
  const mensajeVacio = document.getElementById("mensajeVacio");

  // Si no existe la tabla en la página, se detiene.
  if (!tablaHistorial) return;

  // Obtiene el usuario guardado.
  const usuario = JSON.parse(localStorage.getItem("usuarioPokemonBank"));

  // Si no existe usuario, vuelve al login.
  if (!usuario) {
    window.location.href = "index.html";
    return;
  }

  // Limpia la tabla antes de volver a llenarla.
  tablaHistorial.innerHTML = "";

  // Si el historial no existe o está vacío,
  // muestra un mensaje de vacío.
  if (!usuario.historial || usuario.historial.length === 0) {
    mensajeVacio.classList.remove("d-none");
    return;
  }

  // Si sí hay transacciones, oculta el mensaje vacío.
  mensajeVacio.classList.add("d-none");

  // Recorre el historial una por una.
  usuario.historial.forEach((transaccion, index) => {
    // Va agregando una fila nueva por cada movimiento.
    tablaHistorial.innerHTML += `
      <tr>
        <td>${index + 1}</td>
        <td>${transaccion.tipo}</td>
        <td>$${Number(transaccion.monto).toFixed(2)}</td>
      </tr>
    `;
  });
}

// ==========================================================
// FUNCIÓN: MOSTRAR GRÁFICO
```

```
// ==========================================================
// Esta función cuenta cuántos depósitos, retiros y pagos
// existen en el historial y luego crea un gráfico.
function mostrarGrafico() {
  // Busca el canvas donde se dibuja el gráfico.
  const canvas = document.getElementById("graficoTransacciones");

  // Si el canvas no existe, se detiene.
  if (!canvas) return;

  // Obtiene el usuario actual.
  const usuario = obtenerUsuario();

  // Si no existe usuario, vuelve al login.
  if (!usuario) {
    window.location.href = "index.html";
    return;
  }

  // Variables para contar cuántas transacciones hay
  // de cada tipo.
  let depositos = 0;
  let retiros = 0;
  let pagos = 0;

  // Recorre cada transacción del historial.
  usuario.historial.forEach((transaccion) => {
    // Si el tipo es depósito, suma uno.
    if (transaccion.tipo === "Depósito") {
      depositos++;

    // Si el tipo es retiro, suma uno.
    } else if (transaccion.tipo === "Retiro") {
      retiros++;

    // Si el texto inicia con "Pago de servicio",
    // cuenta esa transacción como pago.
    } else if (transaccion.tipo.startsWith("Pago de servicio")) {
      pagos++;
    }
  });

  // Estos arreglos serán usados por Chart.js.
  const labels = [];
  const data = [];
  const backgroundColor = [];
  const borderColor = [];
```

```
// Si hubo depósitos, agrega esa categoría.
if (depositos > 0) {
  labels.push("Depósitos");
  data.push(depositos);
  backgroundColor.push("#198754");
  borderColor.push("#157347");
}

// Si hubo retiros, agrega esa categoría.
if (retiros > 0) {
  labels.push("Retiros");
  data.push(retiros);
  backgroundColor.push("#ffc107");
  borderColor.push("#e0a800");
}

// Si hubo pagos, agrega esa categoría.
if (pagos > 0) {
  labels.push("Pagos de servicios");
  data.push(pagos);
  backgroundColor.push("#0dcaf0");
  borderColor.push("#0aa2c0");
}

// Si no hay datos, no se crea el gráfico.
if (data.length === 0) {
  return;
}

// Se crea el gráfico de dona.
const graficoTransacciones = new Chart(canvas, {
  type: "doughnut",
  data: {
    labels: labels,
    datasets: [{
      label: "Cantidad de transacciones",
      data: data,
      backgroundColor: backgroundColor,
      borderColor: borderColor,
      borderWidth: 2
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: {
        position: "bottom"
```

```
        },
        title: {
          display: true
        }
      }
    }
  });

  window.addEventListener("resize", () => {
    graficoTransacciones.resize();
  });
}

// ==========================================================
// CUANDO LA PÁGINA TERMINA DE CARGAR
// ==========================================================
// Este evento espera a que el HTML esté listo
// y luego ejecuta las funciones necesarias.
document.addEventListener("DOMContentLoaded", () => {
  // Carga nombre y cuenta si esos elementos existen.
  cargarDatosUsuario();

  // Muestra historial si la tabla está en esa página.
  mostrarHistorial();

  // Muestra gráfico si existe el canvas.
  mostrarGrafico();
});

// ==========================================================
// EVENTOS DE LOS MODALES
// ==========================================================
// Cuando se abre cada modal, el cursor se coloca
// automáticamente en el campo más útil.

// Al abrir el modal de depósito...
$('#modalDepositar').on('shown.bs.modal', function () {
  // ...enfoca el input del monto.
  $('#montoDeposito').trigger('focus');
});

// Al abrir el modal de retiro...
$('#modalRetirar').on('shown.bs.modal', function () {
  // ...enfoca el input del monto.
  $('#montoRetiro').trigger('focus');
});

// Al abrir el modal de pago de servicios...
```

```
$('#modalPagoServicio').on('shown.bs.modal', function () {
  // ...enfoca el select del servicio.
  $('#tipoServicio').trigger('focus');
});
// Fin main.js
```

css/styles.css

```
/* ==========================================================
   ESTILOS GENERALES
   ========================================================== */
/* Estilo general del body.
   Define la fuente, elimina márgenes
   y aplica un fondo degradado claro. */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: linear-gradient(135deg, #e9edf1, #d8dee5);
}

/* Todas las tarjetas tendrán bordes redondeados. */
.card {
  border-radius: 20px;
}

/* Todos los botones tendrán bordes redondeados
   y letra un poco más fuerte. */
.btn {
  border-radius: 12px;
  font-weight: 600;
}

/* Ajusta el estilo general de las tablas. */
.table {
  margin-bottom: 0;
  border-radius: 12px;
  overflow: hidden;
}

/* Quita el borde inferior de los encabezados de tabla. */
.table thead th {
  border-bottom: 0 !important;
}

/* ==========================================================
   LOGIN
   ========================================================== */
```

```
/* Contenedor del login centrado vertical y horizontalmente. */
.login-wrapper {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}

/* Tarjeta del login. */
.login-card {
  width: 100%;
  max-width: 500px;
  border-radius: 20px;
  border: none;
}

/* Botón principal del login. */
.btn-login-custom {
  background-color: #2b4ea2;
  color: white;
  border: none;
}

/* Hover del botón de login. */
.btn-login-custom:hover {
  background-color: #1f3c84;
  color: white;
}

/* ========================================================
   LOGO
   ======================================================== */
/* Contenedor del logo centrado. */
.bank-logo-wrap {
  display: flex;
  justify-content: center;
  margin-bottom: 22px;
}

/* Imagen del logo. */
.bank-logo {
  width: 260px;
  max-width: 100%;
  height: auto;
  display: block;
}
```

```
/* ============================================================
   TARJETA PRINCIPAL
   ============================================================ */
/* Tarjeta principal de la pantalla de acciones. */
.main-card {
  width: 100%;
  max-width: 500px;
  border-radius: 28px;
  border: none;
  background-color: white;
  padding: 24px;
  overflow: hidden;
}

/* Parte superior donde va el logo y datos del usuario. */
.top-section {
  background-color: #f8f9fa;
  border-radius: 24px;
  padding: 28px 28px 24px;
  box-shadow: 0 12px 26px rgba(0, 0, 0, 0.22);
  margin: -24px -24px 28px -24px;
  position: relative;
  z-index: 2;
}

/* Sección de los botones principales. */
.actions-section {
  position: relative;
  z-index: 1;
}

/* Ajuste de margen del bloque de botones. */
.action-buttons {
  margin-top: 0;
}

/* Espaciado lateral de cada columna de botón. */
.action-buttons .col-md-6 {
  padding-left: 15px;
  padding-right: 15px;
}

/* ============================================================
   PANEL DEL USUARIO
   ============================================================ */
/* Espacio superior del panel del usuario. */
.user-panel {
  margin-top: 10px;
```

```
}

/* Imagen redonda del usuario. */
.user-avatar {
  width: 65px;
  height: 65px;
  object-fit: cover;
  border-radius: 50%;
  border: 3px solid #2b4ea2;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.15);
}

/* Texto del nombre y la cuenta. */
.user-data-box p {
  margin: 0;
  padding-left: 15px;
  font-size: 15px;
  line-height: 1.5;
}

/* Botón de cerrar sesión. */
.btn-logout-custom {
  background-color: #dc3545;
  color: white;
  min-height: 52px;
  border-radius: 16px;
  font-size: 16px;
  font-weight: 700;
  white-space: nowrap;
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  text-align: center;
  border: none;
}

/* Hover del botón de cerrar sesión. */
.btn-logout-custom:hover {
  background-color: #c82333;
  color: white;
}

/* ========================================================
   BOTONES PRINCIPALES
   ======================================================== */
/* Estilo base de todos los botones grandes del banco. */
.btn-bank {
```

```
  border: none;
  border-radius: 16px;
  padding: 14px 20px;
  font-size: 18px;
  font-weight: 700;
  color: white;
  transition: all 0.25s ease;
  box-shadow: 0 6px 14px rgba(0, 0, 0, 0.12);
}

/* Efecto hover general. */
.btn-bank:hover {
  transform: translateY(-2px);
  box-shadow: 0 10px 18px rgba(0, 0, 0, 0.18);
  color: white;
}

/* Quita borde azul al enfocar. */
.btn-bank:focus {
  outline: none;
  box-shadow: none;
}

/* Botón depositar. */
.btn-depositar {
  background-color: #ffcc33;
  color: white;
}

/* Hover depositar. */
.btn-depositar:hover {
  background-color: #e6b82e;
  color: white;
}

/* Botón retirar. */
.btn-retirar {
  background-color: #ffa832;
  color: white;
}

/* Hover retirar. */
.btn-retirar:hover {
  background-color: #c96a16;
  color: white;
}

/* Botón consultar saldo. */
```

```
.btn-saldo {
  background-color: #fe8330;
  color: white;
}

/* Hover consultar saldo. */
.btn-saldo:hover {
  background-color: #e87422;
  color: white;
}

/* Botón pagar servicios. */
.btn-servicios {
  background-color: #fe5f2f;
  color: white;
}

/* Hover pagar servicios. */
.btn-servicios:hover {
  background-color: #e55328;
  color: white;
}

/* Botón historial. */
.btn-historial {
  background-color: #2b4ea2;
  color: white;
}

/* Hover historial. */
.btn-historial:hover {
  background-color: #1f3c84;
  color: white;
}

/* Botón gráfico. */
.btn-grafico {
  background-color: white;
  color: #2b4ea2;
}

/* Hover gráfico. */
.btn-grafico:hover {
  background-color: #f1f1f1;
  color: #2b4ea2;
}

/* ========================================================
```

```
   MODALES
   ========================================================== */
/* Clase opcional para poner el signo $ en negrita. */
.dolar-bold {
  font-weight: 700;
}

/* Pone en negrita todos los label dentro de modales. */
.modal-body label {
  font-weight: 700;
}

/* Ancho máximo del modal. */
.modal-form {
  max-width: 500px;
}

/* Caja principal del modal. */
.modal-content {
  border-radius: 18px;
  overflow: hidden;
  border: none;
}

/* Encabezado azul del modal. */
.modal-header-bank {
  background-color: #0d6efd;
  color: white;
  position: relative;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 14px 20px;
  border-bottom: none;
}

/* Título del modal. */
.modal-title {
  font-size: 20px;
  font-weight: bold;
}

/* Botón de cerrar del encabezado. */
.modal-close-btn {
  position: absolute;
  right: 0;
  top: 0;
  width: 46px;
```

```
  height: 100%;
  border: none;
  background: transparent;
  color: white;
  font-size: 18px;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.2s ease;
}

/* Hover del botón cerrar modal. */
.modal-close-btn:hover {
  background-color: #c42b1c;
}

/* Quita contorno al enfocar. */
.modal-close-btn:focus {
  outline: none;
}

/* Da espacio entre botones del footer. */
.modal-footer .btn + .btn {
  margin-left: 40px;
}

/* Estilo general de botones del modal. */
.btn-modal-confirm,
.btn-modal-cancel {
  min-width: 130px;
  border: none;
  border-radius: 12px;
  font-weight: 700;
  padding: 10px 22px;
  transition: all 0.25s ease;
}

/* Botón confirmar del modal. */
.btn-modal-confirm {
  background-color: #0d6efd;
  color: white;
}

/* Hover confirmar. */
.btn-modal-confirm:hover {
  background-color: #0b5ed7;
  color: white;
}
```

```
/* Botón cancelar del modal. */
.btn-modal-cancel {
  background-color: #6c757d;
  color: white;
}

/* Hover cancelar. */
.btn-modal-cancel:hover {
  background-color: #5a6268;
  color: white;
}

/* ==========================================================
   SWEETALERT
   ========================================================== */
/* Caja general de SweetAlert. */
.swal-modal {
  border-radius: 18px;
}

/* Título centrado. */
.swal-title {
  text-align: center;
}

/* Texto centrado. */
.swal-text {
  margin-top: 12px;
  text-align: center;
  color: gray;
}

/* Espacio lateral entre botones. */
.swal-button-container {
  margin: 0 15px;
}

/* Botón confirmar SweetAlert. */
.swal-button--confirm {
  background-color: #0d6efd;
}

/* Hover confirmar SweetAlert. */
.swal-button--confirm:hover {
  background-color: #0b5ed7;
}

/* Botón cancelar SweetAlert. */
```

```
.swal-button--cancel {
  background-color: #6c757d;
}

/* Footer de SweetAlert. */
.swal-footer {
  margin-top: 12px;
  text-align: center;
}

/* ==========================================================
   HISTORIAL
   ========================================================== */
/* Tarjeta principal del historial. */
.historial-card {
  border-radius: 20px;
  overflow: hidden;
  border: none;
}

/* Encabezado del historial. */
.historial-header {
  margin: 0;
  background-color: #2b4ea2;
  color: white;
  padding: 24px 28px;
  border-radius: 0;
  position: relative;
  z-index: 2;
  box-shadow: 0 10px 0 #1f3c84, 0 16px 22px rgba(0, 0, 0, 0.18);
}

/* Área donde está la tabla. */
.historial-table-section {
  padding: 26px 28px 28px;
}

/* Encabezado azul oscuro de la tabla. */
.table-head-custom {
  background-color: #1e3c7a;
}

/* Columnas del encabezado. */
.table-head-custom th {
  background-color: #1e3c7a;
  color: white;
  border-top: none;
  border-bottom: 0 !important;
```

```
  border-left: 1px solid rgba(255, 255, 255, 0.18);
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  padding-top: 16px;
  padding-bottom: 16px;
  box-shadow: inset 0 -3px 0 #142a57;
}

/* ========================================================
   GRÁFICO
   ======================================================== */
/* Tarjeta del gráfico. */
.grafico-card {
  border-radius: 20px;
  overflow: hidden;
  border: none;
}

/* Encabezado del gráfico. */
.grafico-header {
  margin: 0;
  background-color: white;
  color: #2b4ea2;
  padding: 24px 28px;
  border-radius: 0;
  position: relative;
  z-index: 2;
  box-shadow: 0 8px 0 #e3e7f2, 0 14px 20px rgba(0, 0, 0, 0.1);
}

/* Título del gráfico. */
.grafico-header h2 {
  margin: 0;
  color: #2b4ea2;
  font-weight: bold;
}

/* Contenedor interno del gráfico. */
.grafico-content-section {
  padding: 26px 28px 28px;
}

/* Área máxima del canvas. */
.chart-container {
  width: 100%;
  max-width: 850px;
  height: 420px;
  margin: 0 auto;
}
```

```
/* ==========================================================
   BOTÓN VOLVER
   ========================================================== */
/* Botón volver usado en historial y gráfico. */
.btn-volver-custom {
  background-color: #dc3545;
  color: white;
  border: none;
  border-radius: 12px;
  font-weight: 700;
  padding: 10px 20px;
}

/* Hover del botón volver. */
.btn-volver-custom:hover {
  background-color: #c82333;
  color: white;
}

/* ==========================================================
   RESPONSIVE
   ========================================================== */
/* Ajustes para pantallas pequeñas. */
@media (max-width: 768px) {
  .user-avatar {
    width: 60px;
    height: 60px;
  }

  .user-data-box p {
    font-size: 16px;
  }

  .chart-container {
    max-width: 100%;
    height: 260px;
  }

  .btn-logout-custom {
    min-height: 56px;
    font-size: 16px;
  }
}
/*Fin styles.css*/
```

images/

AshKetchumPFP.jpg	PKBC.png	PKBL.png

index.hmtl

```
<!DOCTYPE html>
<html lang="es">
<head>
  <!-- Define codificación UTF-8 para acentos y símbolos. -->
  <meta charset="UTF-8" />

  <!-- Hace que la página se vea bien en celular. -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <!-- Título de la pestaña. -->
  <title>Login</title>

  <!-- Icono de la pestaña del navegador. -->
  <link rel="icon" type="image/png" href="images/PKBL.png">

  <!-- Hoja de estilos de Bootstrap. -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css"
/>

  <!-- Hoja de estilos personalizada del proyecto. -->
  <link rel="stylesheet" href="./css/styles.css" />
</head>
<body class="bg-light">

  <!-- Contenedor principal del login. -->
  <div class="login-wrapper">
    <div class="card shadow p-4 login-card">

      <!-- Logo principal del banco. -->
```

```html
      <div class="bank-logo-wrap mb-3">
        <img src="images/PKBC.png" alt="Pokémon Bank" class="bank-logo">
      </div>

      <!-- Campo donde el usuario escribe el PIN. -->
      <div class="mb-3">
        <strong><label for="pin" class="form-label">PIN:</label></strong>
        <input
          type="password"
          id="pin"
          class="form-control"
          maxlength="4"
          placeholder="****"
          inputmode="numeric"
        />
      </div>

      <!-- Botón que ejecuta la función iniciarSesion(). -->
      <button class="btn btn-login-custom w-100" onclick="iniciarSesion()">
        Ingresar
      </button>
    </div>
  </div>

  <!-- Librería jQuery. -->
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>

  <!-- Librería Popper, necesaria para ciertos componentes Bootstrap. -->
  <script
src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"></s
cript>

  <!-- JavaScript de Bootstrap. -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.min.js"></
script>

  <!-- Librería SweetAlert para alertas bonitas. -->
  <script src="https://unpkg.com/sweetalert/dist/sweetalert.min.js"></script>

  <!-- Archivo principal de lógica JavaScript. -->
  <script src="./js/main.js"></script>
</body>
</html>
<!-- Fin index.html -->
```

acciones.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <!-- Configuración básica de caracteres y responsive. -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Título de la página. -->
  <title>Acciones</title>

  <!-- Icono de pestaña. -->
  <link rel="icon" type="image/png" href="images/PKBL.png">

  <!-- Bootstrap. -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css"
>

  <!-- Estilos personalizados. -->
  <link rel="stylesheet" href="./css/styles.css">
</head>
<body class="bg-light">

  <!-- Contenedor principal. -->
  <div class="container py-5">
    <div class="card shadow main-card mx-auto">

      <!-- Encabezado con logo y datos del usuario. -->
      <section class="top-section">
        <div class="bank-logo-wrap">
          <img src="images/PKBC.png" alt="Pokémon Bank" class="bank-logo">
        </div>

        <div class="user-panel">
          <div class="row align-items-center">
            <div class="col-3 col-sm-2 col-md-2 text-center">
              <img src="images/AshKetchumPFP.jpg" alt="Foto de Ash"
class="user-avatar">
            </div>

            <div class="col-9 col-sm-6 col-md-6">
              <div class="user-data-box">
                <!-- Aquí se inserta el nombre desde JavaScript. -->
                <p><strong>Usuario:</strong> <span
id="nombreUsuario"></span></p>
```

```
                <!-- Aquí se inserta el número de cuenta desde JavaScript. -->
                <p><strong>Cuenta:</strong> <span
id="numeroCuenta"></span></p>
              </div>
            </div>

            <div class="col-12 col-sm-4 col-md-4 mt-3 mt-sm-0">
              <!-- Botón para cerrar sesión. -->
              <button class="btn btn-logout-custom btn-block"
onclick="cerrarSesion()">
                Cerrar sesión
              </button>
            </div>
          </div>
        </div>
      </section>

      <!-- Botones principales de operaciones. -->
      <section class="actions-section">
        <div class="row action-buttons">

          <!-- Abre el modal de depósito. -->
          <div class="col-md-6 mb-3">
            <button class="btn btn-bank btn-depositar w-100" data-
toggle="modal" data-target="#modalDepositar">
              Depositar
            </button>
          </div>

          <!-- Abre el modal de retiro. -->
          <div class="col-md-6 mb-3">
            <button class="btn btn-bank btn-retirar w-100" data-toggle="modal"
data-target="#modalRetirar">
              Retirar
            </button>
          </div>

          <!-- Ejecuta la consulta de saldo. -->
          <div class="col-md-6 mb-3">
            <button class="btn btn-bank btn-saldo w-100"
onclick="consultarSaldo()">
              Consultar saldo
            </button>
          </div>

          <!-- Abre el modal de pago de servicios. -->
          <div class="col-md-6 mb-3">
```

```
            <button class="btn btn-bank btn-servicios w-100" data-
toggle="modal" data-target="#modalPagoServicio">
              Pagar servicios
            </button>
          </div>

          <!-- Enlace al historial. -->
          <div class="col-md-6 mb-3">
            <a href="historial.html" class="btn btn-bank btn-historial w-100">
              Ver historial
            </a>
          </div>

          <!-- Enlace al gráfico. -->
          <div class="col-md-6 mb-3">
            <a href="grafico.html" class="btn btn-bank btn-grafico w-100">
              Ver gráfico
            </a>
          </div>
        </div>
      </section>
    </div>
  </div>

  <!-- MODAL DE DEPÓSITO -->
  <div class="modal fade" id="modalDepositar" tabindex="-1" aria-
labelledby="modalDepositarLabel" aria-hidden="true">
    <div class="modal-dialog modal-form">
      <div class="modal-content">
        <div class="modal-header modal-header-bank justify-content-center
position-relative">
          <h5 class="modal-title m-0" id="modalDepositarLabel">Depósito</h5>
          <button type="button" class="modal-close-btn" data-dismiss="modal"
aria-label="Cerrar">✕</button>
        </div>

        <div class="modal-body">
          <form id="formDepositar">
            <div class="form-group">
              <label for="montoDeposito">Monto a depositar:</label>

              <div class="input-group">
                <div class="input-group-prepend">
                  <span class="input-group-text dolar-bold">$</span>
                </div>

                <input
                  type="number"
```

```
                  id="montoDeposito"
                  class="form-control"
                  min="1"
                  step="0.01"
                  placeholder="Ingrese el monto"
                >
              </div>
            </div>
          </form>
        </div>

        <div class="modal-footer justify-content-center">
          <button type="button" class="btn btn-modal-confirm"
onclick="procesarDeposito()">Depositar</button>
          <button type="button" class="btn btn-modal-cancel" data-
dismiss="modal">Cancelar</button>
        </div>
      </div>
    </div>
  </div>

  <!-- MODAL DE RETIRO -->
  <div class="modal fade" id="modalRetirar" tabindex="-1" aria-
labelledby="modalRetirarLabel" aria-hidden="true">
    <div class="modal-dialog modal-form">
      <div class="modal-content">
        <div class="modal-header modal-header-bank justify-content-center
position-relative">
          <h5 class="modal-title m-0" id="modalRetirarLabel">Retiro</h5>
          <button type="button" class="modal-close-btn" data-dismiss="modal"
aria-label="Cerrar">X</button>
        </div>

        <div class="modal-body">
          <form id="formRetirar">
            <div class="form-group">
              <label for="montoRetiro">Monto a retirar:</label>

              <div class="input-group">
                <div class="input-group-prepend">
                  <span class="input-group-text dolar-bold">$</span>
                </div>

                <input
                  type="number"
                  id="montoRetiro"
                  class="form-control"
                  min="1"
```

```
                  step="0.01"
                  placeholder="Ingrese el monto"
                >
              </div>
            </div>
          </form>
        </div>

        <div class="modal-footer justify-content-center">
          <button type="button" class="btn btn-modal-confirm"
onclick="procesarRetiro()">Retirar</button>
          <button type="button" class="btn btn-modal-cancel" data-
dismiss="modal">Cancelar</button>
        </div>
      </div>
    </div>
  </div>

  <!-- MODAL DE PAGO DE SERVICIOS -->
  <div class="modal fade" id="modalPagoServicio" tabindex="-1" aria-
labelledby="modalPagoServicioLabel" aria-hidden="true">
    <div class="modal-dialog modal-form">
      <div class="modal-content">
        <div class="modal-header modal-header-bank justify-content-center
position-relative">
          <h5 class="modal-title m-0" id="modalPagoServicioLabel">Pago de
servicios</h5>
          <button type="button" class="modal-close-btn" data-dismiss="modal"
aria-label="Cerrar">✕</button>
        </div>

        <div class="modal-body">
          <form id="formPagoServicio">
            <div class="form-group">
              <label for="tipoServicio">Seleccione el servicio:</label>
              <select id="tipoServicio" class="form-control">
                <option value="">Seleccione una opción</option>
                <option value="Agua potable">Agua potable</option>
                <option value="Energía Eléctrica">Energía Eléctrica</option>
                <option value="Internet">Internet</option>
                <option value="Telefonía">Telefonía</option>
              </select>
            </div>

            <div class="form-group mt-3">
              <label for="montoServicio">Monto a pagar:</label>

              <div class="input-group">
```

```
                <div class="input-group-prepend">
                  <span class="input-group-text dolar-bold">$</span>
                </div>

                <input
                  type="number"
                  id="montoServicio"
                  class="form-control"
                  min="1"
                  step="0.01"
                  placeholder="Ingrese el monto"
                >
              </div>
            </div>
          </form>
        </div>

        <div class="modal-footer justify-content-center">
          <button type="button" class="btn btn-modal-confirm"
onclick="procesarPagoServicio()">Pagar</button>
          <button type="button" class="btn btn-modal-cancel" data-
dismiss="modal">Cancelar</button>
        </div>
      </div>
    </div>
  </div>

  <!-- Librerías JS. -->
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
  <script
src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"></s
cript>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.min.js"></
script>
  <script src="https://unpkg.com/sweetalert/dist/sweetalert.min.js"></script>

  <!-- Script principal. -->
  <script src="./js/main.js"></script>
</body>
</html>
<!-- Fin acciones.html -->
```

historial.html

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
  <!-- Configuración básica. -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Título de la pestaña. -->
  <title>Historial</title>

  <!-- Icono. -->
  <link rel="icon" type="image/png" href="images/PKBL.png">

  <!-- Bootstrap. -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css"
>

  <!-- Estilos propios. -->
  <link rel="stylesheet" href="./css/styles.css">
</head>
<body class="bg-light">

  <div class="container py-5">
    <div class="card shadow historial-card">

      <!-- Encabezado principal de la página. -->
      <header class="historial-header">
        <div class="d-flex justify-content-between align-items-center flex-
wrap">
          <h2 class="mb-0">Historial de transacciones</h2>
          <a href="acciones.html" class="btn btn-volver-custom mt-3 mt-sm-
0">Volver</a>
        </div>
      </header>

      <!-- Sección de la tabla. -->
      <section class="historial-table-section">
        <div class="table-responsive">
          <table class="table table-bordered table-striped align-middle">
            <thead class="table-head-custom">
              <tr>
                <th>#</th>
                <th>Tipo</th>
                <th>Monto</th>
              </tr>
            </thead>

            <!-- Aquí JavaScript insertará las filas. -->
            <tbody id="tablaHistorial"></tbody>
```

```
          </table>
        </div>

        <!-- Mensaje que aparece si no hay transacciones. -->
        <p id="mensajeVacio" class="text-center text-muted mt-3 d-none">
          No hay transacciones registradas.
        </p>
      </section>
    </div>
  </div>

  <!-- Librerías. -->
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
  <script
src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"></s
cript>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/bootstrap.min.js"></
script>
  <script src="https://unpkg.com/sweetalert/dist/sweetalert.min.js"></script>

  <!-- Lógica principal. -->
  <script src="./js/main.js"></script>
</body>
</html>
<!-- Fin historial.html -->
```

grafico.html

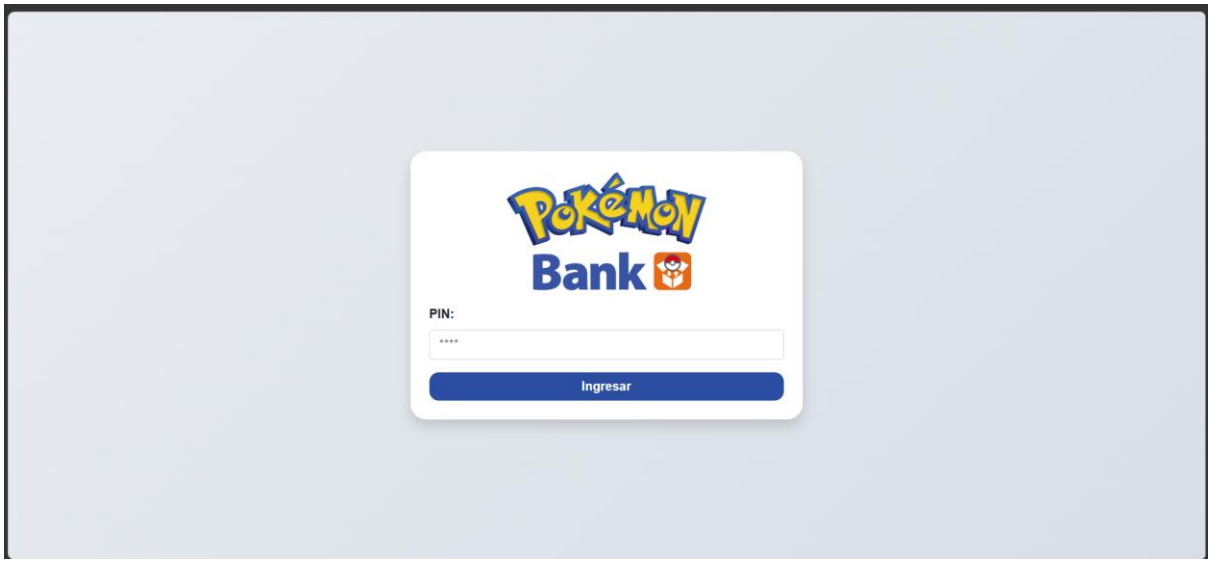
```
<!DOCTYPE html>
<html lang="es">
<head>
  <!-- Configuración básica. -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Título. -->
  <title>Gráfico</title>

  <!-- Icono. -->
  <link rel="icon" type="image/png" href="images/PKBL.png">

  <!-- Bootstrap. -->
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css"
>
```

```
  <!-- Estilos propios. -->
  <link rel="stylesheet" href="./css/styles.css">
</head>
<body class="bg-light">

  <div class="container py-5">
    <div class="card shadow grafico-card">

      <!-- Encabezado de la página. -->
      <header class="grafico-header">
        <div class="d-flex justify-content-between align-items-center flex-
wrap">
          <h2 class="mb-0">Gráfico de transacciones</h2>
          <a href="acciones.html" class="btn btn-volver-custom mt-3 mt-sm-
0">Volver</a>
        </div>
      </header>

      <!-- Zona donde va el gráfico. -->
      <section class="grafico-content-section">
        <div class="chart-container">
          <!-- Aquí Chart.js dibuja el gráfico. -->
          <canvas id="graficoTransacciones"></canvas>
        </div>
      </section>
    </div>
  </div>

  <!-- Librería de Chart.js. -->
  <<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

  <!-- SweetAlert. -->
  <script src="https://unpkg.com/sweetalert/dist/sweetalert.min.js"></script>

  <!-- Script principal. -->
  <script src="./js/main.js"></script>
</body>
</html>
<!-- Fin grafico.html -->
```

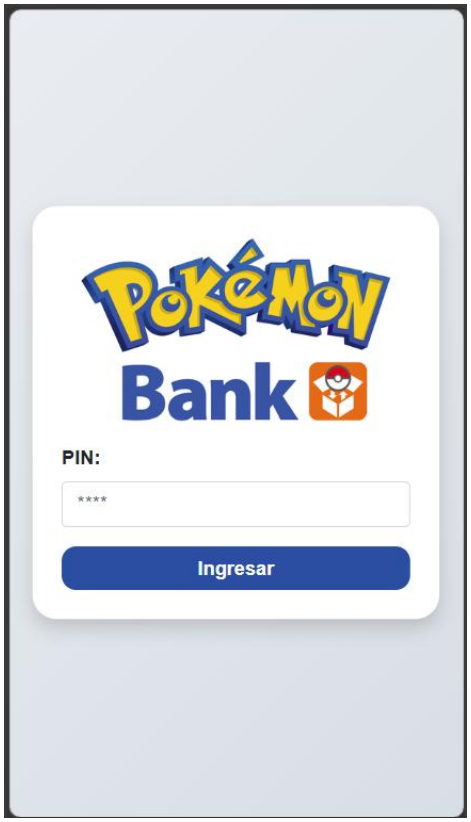
Desarrollo del análisis de proyecto

Pantallas

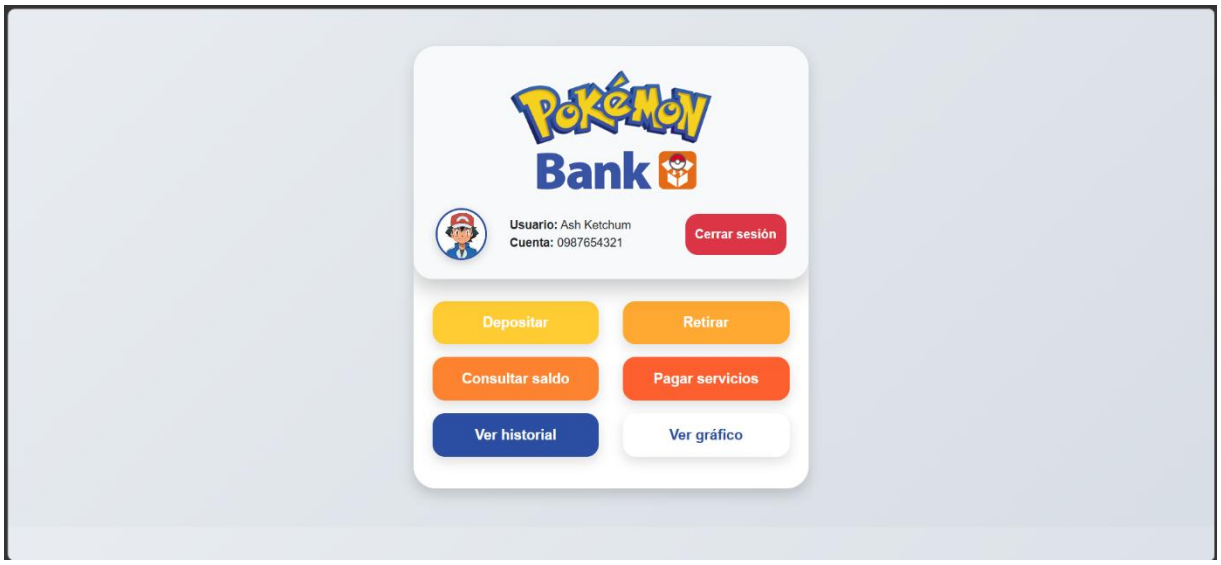
Login



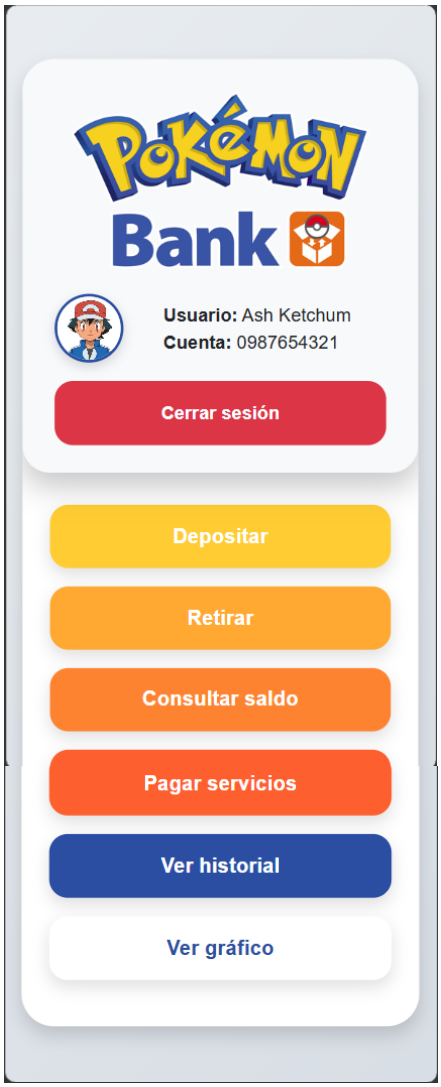
Responsive:



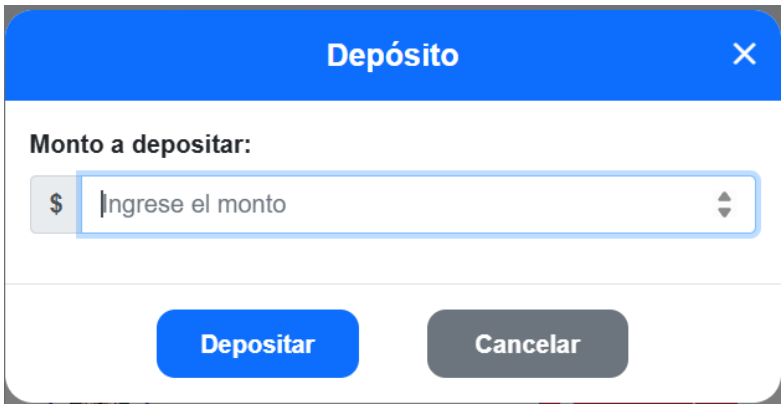
Acciones



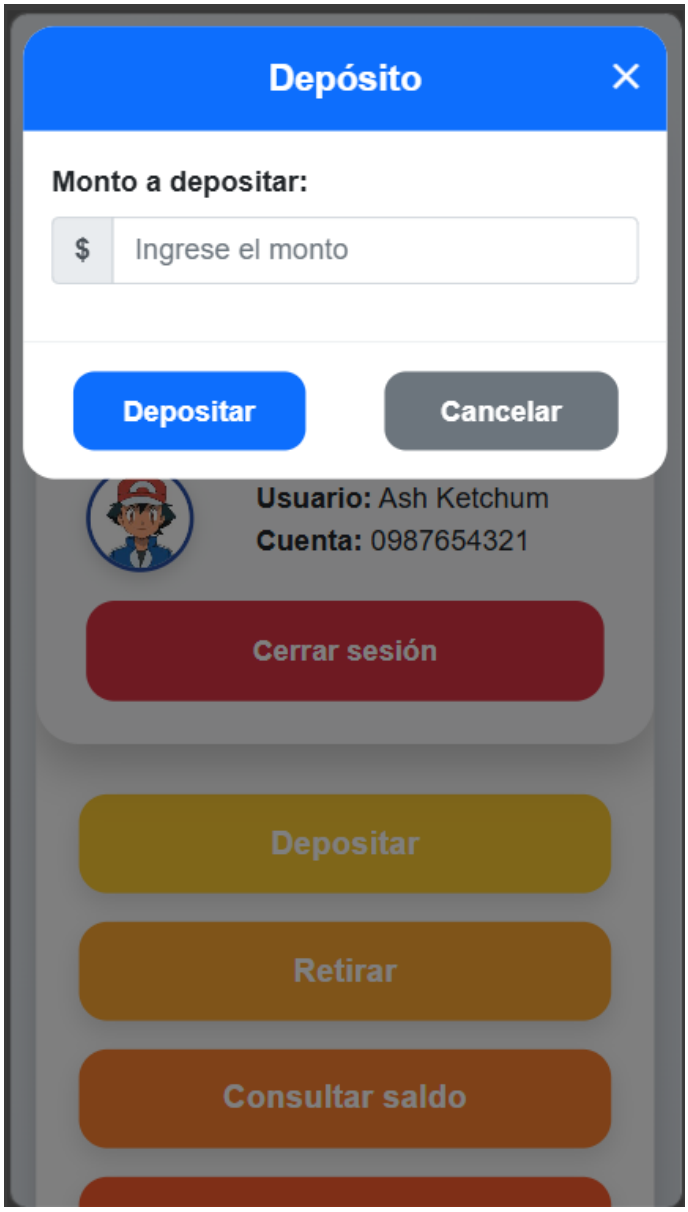
Responsive:



Depositar

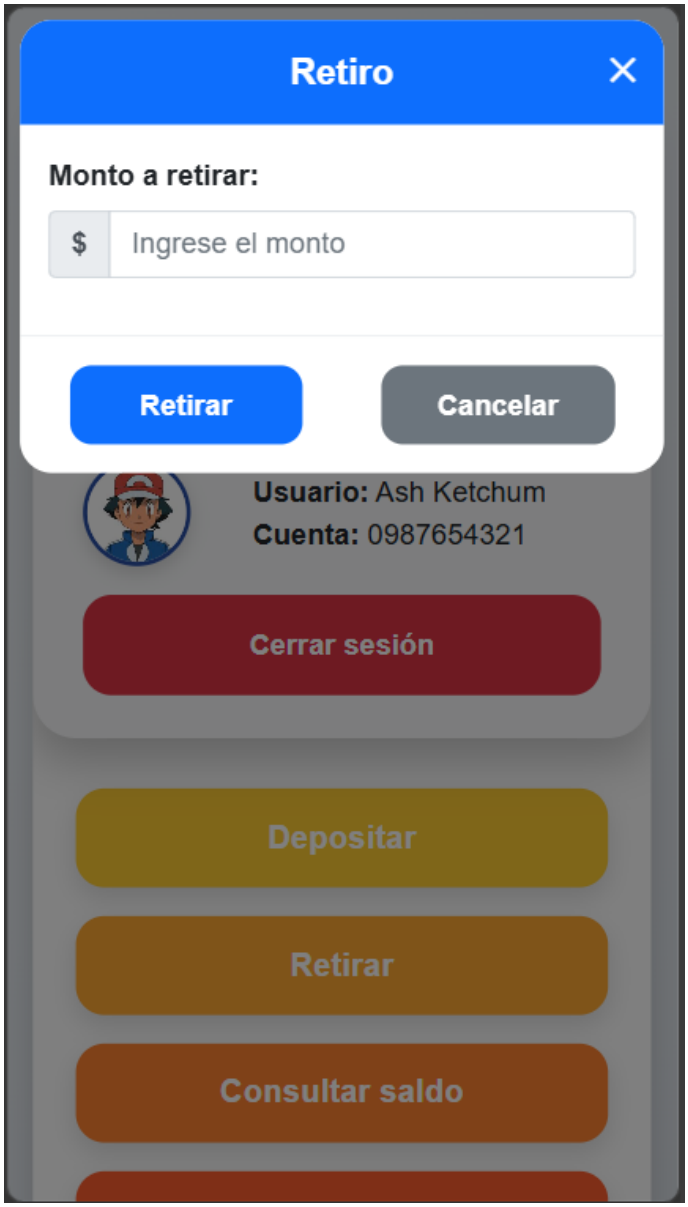


Responsive:

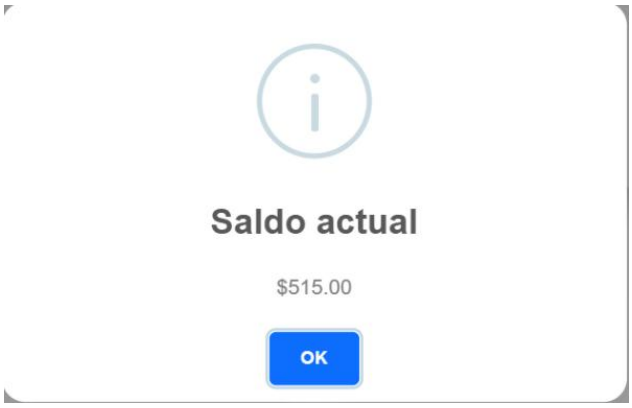


Retirar

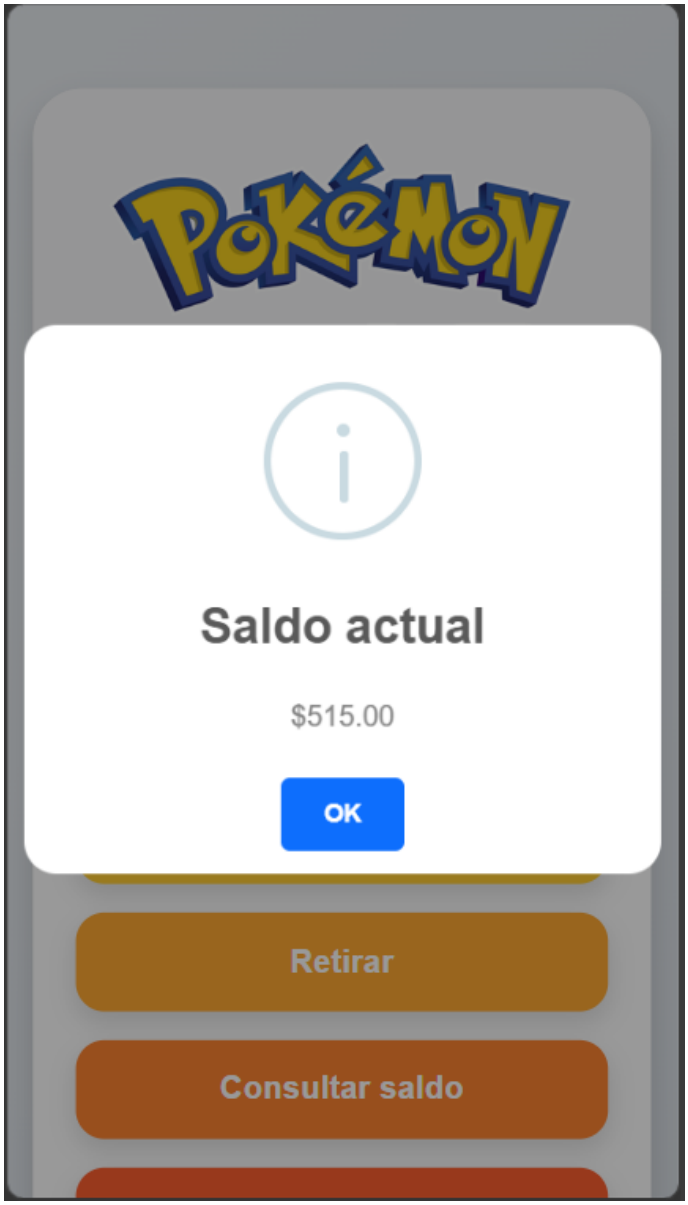
Responsive:



Consultar saldo



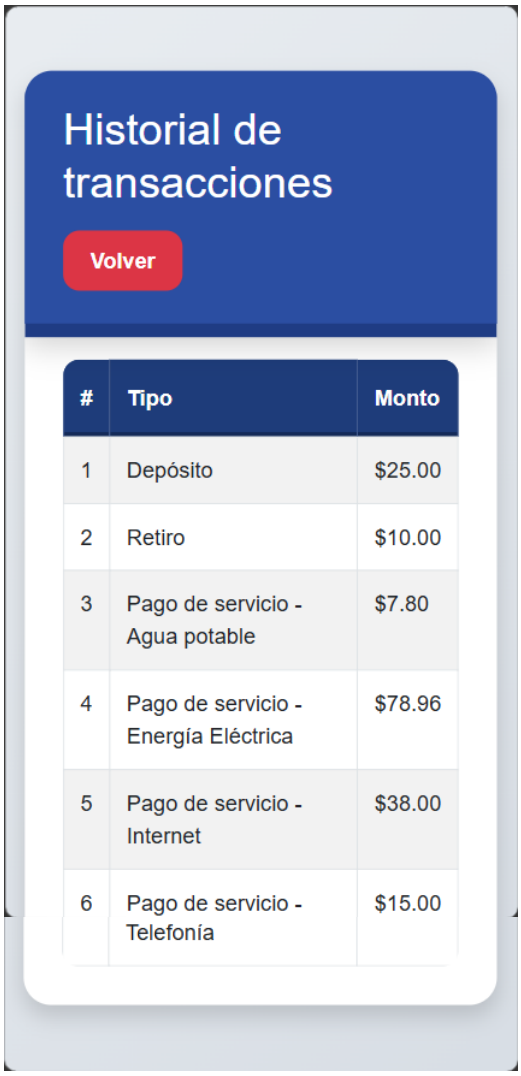
Responsive:



Ver historial

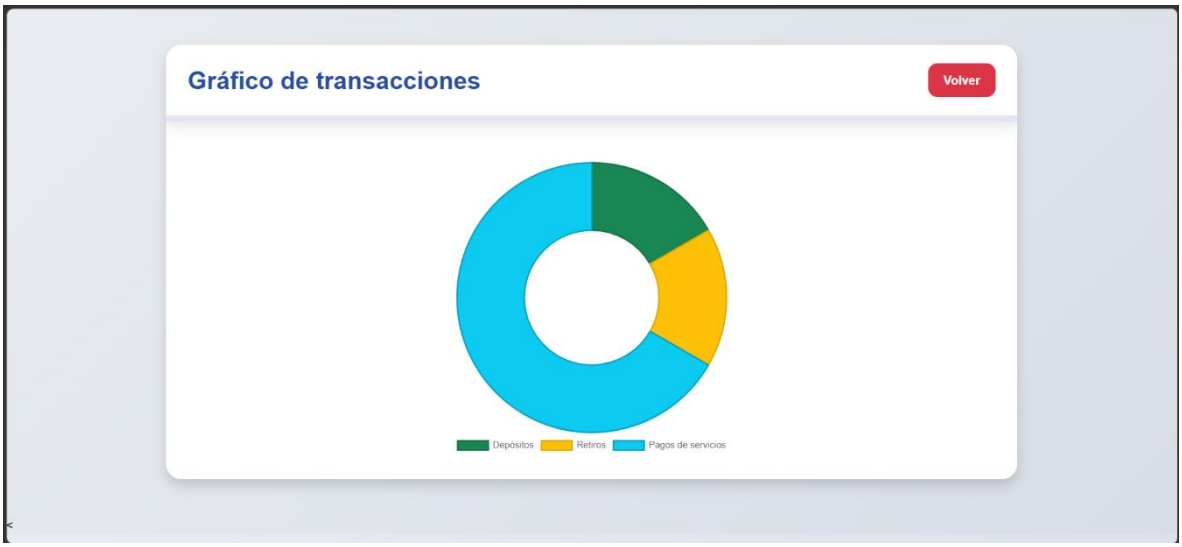
#	Tipo	Monto
1	Depósito	$25.00
2	Retiro	$10.00
3	Pago de servicio - Agua potable	$7.80
4	Pago de servicio - Energía Eléctrica	$78.96
5	Pago de servicio - Internet	$38.00
6	Pago de servicio - Telefonía	$15.00

Responsive:

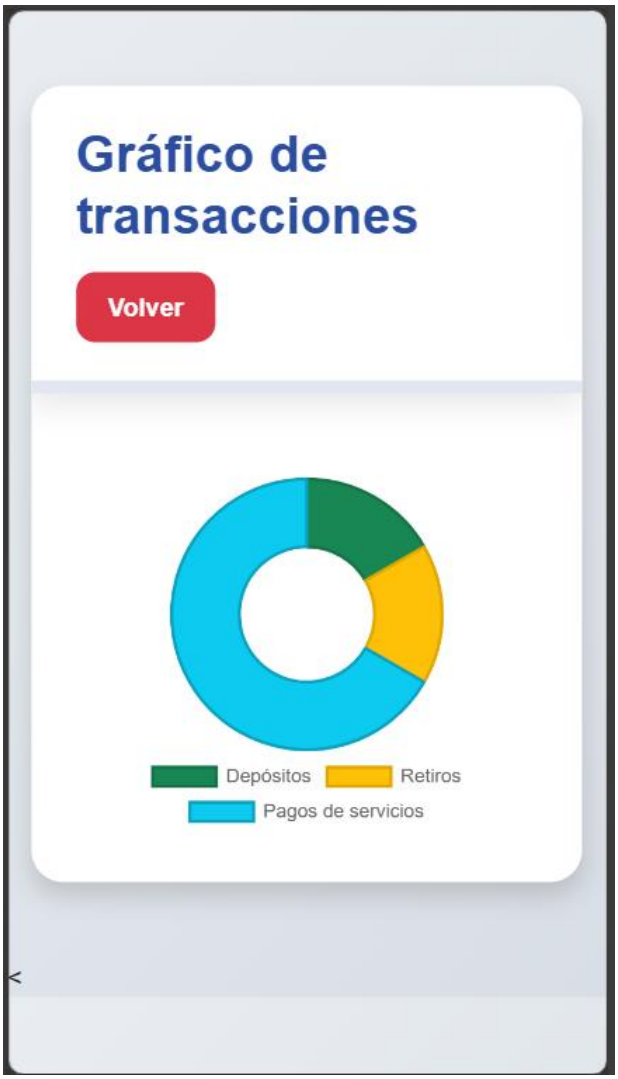


#	Tipo	Monto
1	Depósito	$25.00
2	Retiro	$10.00
3	Pago de servicio - Agua potable	$7.80
4	Pago de servicio - Energía Eléctrica	$78.96
5	Pago de servicio - Internet	$38.00
6	Pago de servicio - Telefonía	$15.00

Ver gráfico



Responsive:

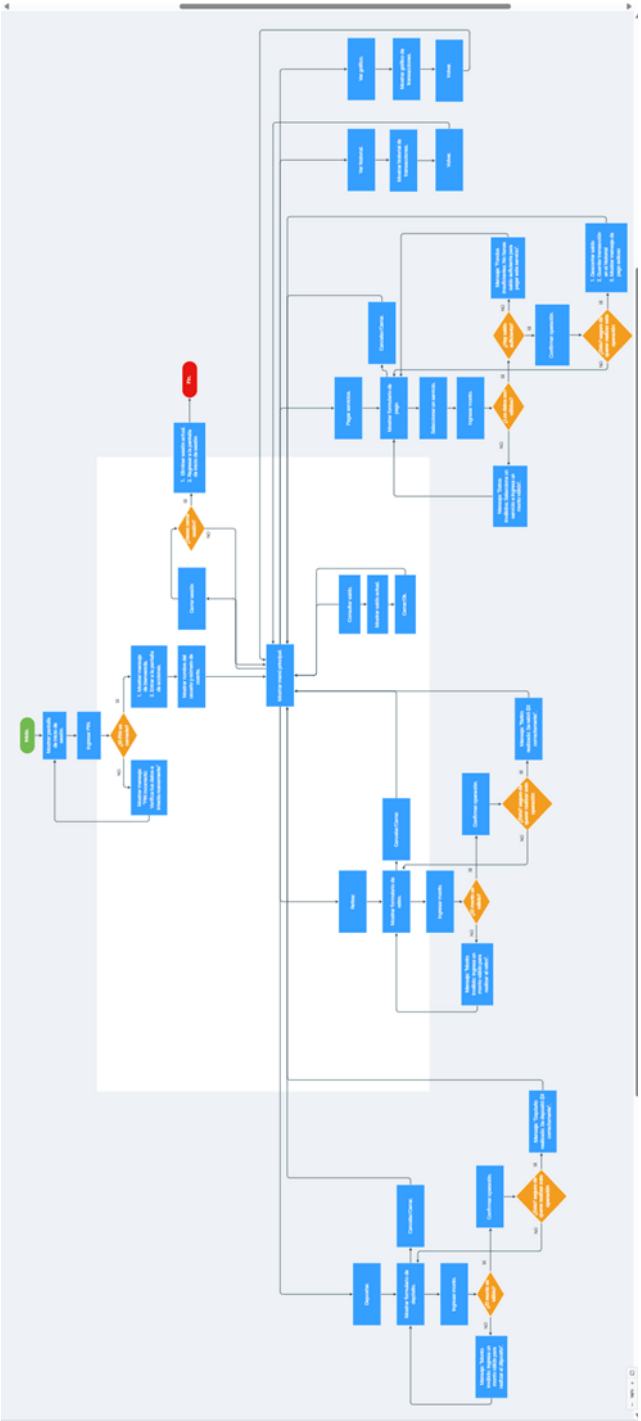


Flujograma

Link:

https://app.moqups.com/WSzIVrLuHANYEMXFUZNGXJeTOGCQpfoA/view/page/ad64222d5?ui=0

Imagen:



Investigación de herramientas

SweetAlert

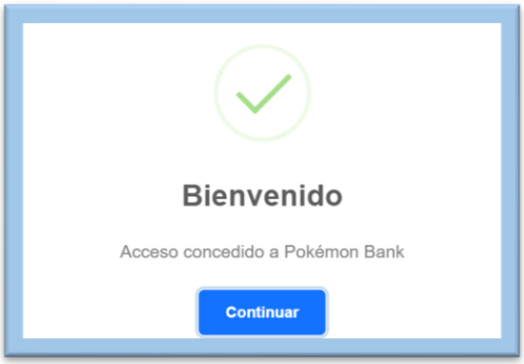
SweetAlert es una librería de JavaScript que permite mostrar mensajes emergentes de una forma más atractiva, ordenada y amigable que las alertas tradicionales del navegador. Su principal ventaja es que mejora la experiencia del usuario al presentar mensajes de éxito, error, advertencia y confirmación con un diseño más profesional.

En el proyecto Pokémon Bank, SweetAlert se utilizó para informar al usuario sobre diferentes acciones importantes dentro del sistema. Por ejemplo, se empleó al momento de iniciar sesión correctamente, mostrando el mensaje de bienvenida “Acceso concedido a Pokémon Bank”. También se utilizó para notificar errores, como cuando el usuario ingresa un PIN incorrecto.

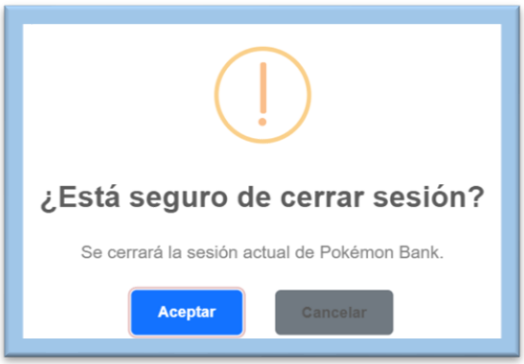
Además, SweetAlert fue implementado en operaciones sensibles que requieren confirmación previa, como cerrar sesión, realizar un depósito, hacer un retiro o pagar un servicio. En estos casos, la librería permitió mostrar una ventana de advertencia antes de ejecutar la acción, ayudando a prevenir errores o movimientos no deseados.

Ejemplos de uso en el proyecto:

- Alerta de éxito al iniciar sesión correctamente.



- Alerta de confirmación antes de cerrar sesión.



- Alerta de advertencia cuando el monto ingresado es inválido.

- Alerta de éxito después de realizar un depósito, retiro o pago de servicio.

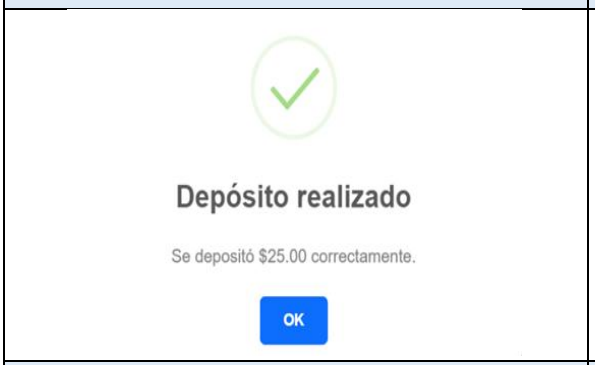
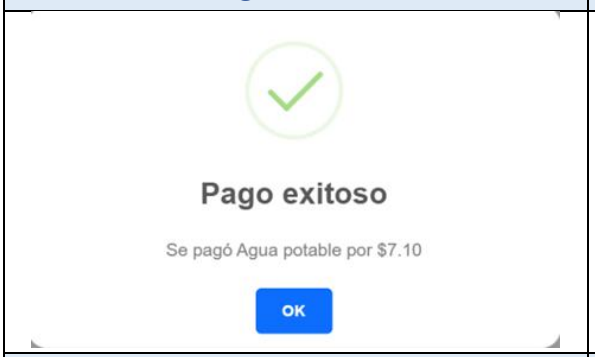
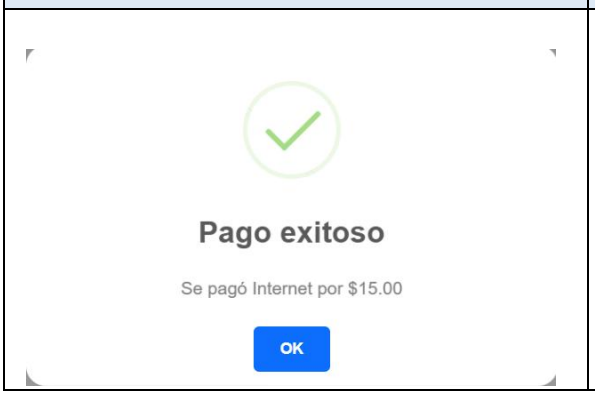
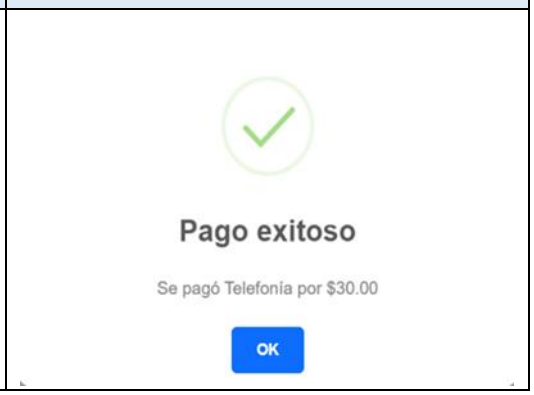
Depósito	Retiro
	
Pago servicios	**Pago servicios**
	
Pago servicios	**Pago servicios**
	

Chart.js

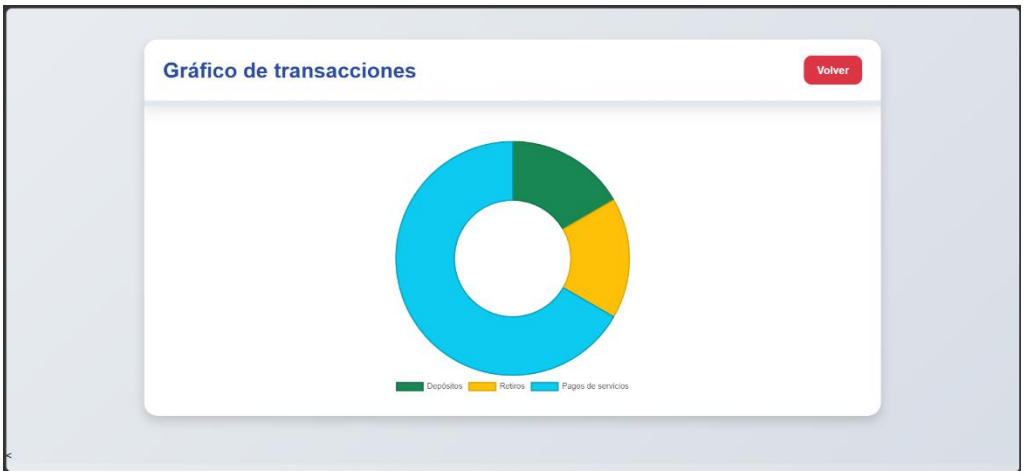
Chart.js es una librería de JavaScript especializada en la creación de gráficos dinámicos e interactivos en páginas web. Su utilidad principal es representar información de manera visual, facilitando la interpretación de datos y permitiendo un análisis más rápido y claro.

En el proyecto Pokémon Bank, Chart.js fue utilizada en la pantalla de gráfico de transacciones, donde el usuario puede observar visualmente el tipo de operaciones que ha realizado dentro del sistema. En este caso, se implementó un gráfico de dona que clasifica las transacciones en depósitos, retiros y pagos de servicios.

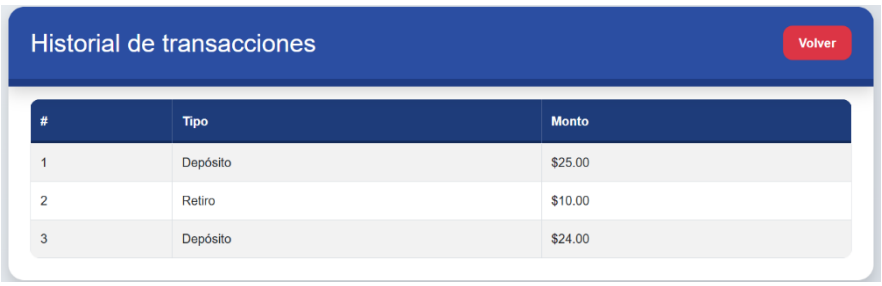
La lógica del gráfico consiste en recorrer el historial del usuario, contar cuántas transacciones hay de cada tipo y luego mostrar solamente las categorías que tengan datos registrados. Esto permite que el gráfico sea más limpio y represente únicamente la actividad real del usuario.

Ejemplos de uso en el proyecto:

- Gráfico de dona para mostrar la cantidad de depósitos, retiros y pagos de servicios realizados.



- Visualización automática de solo las categorías activas, es decir, solo las transacciones que realmente existen en el historial.



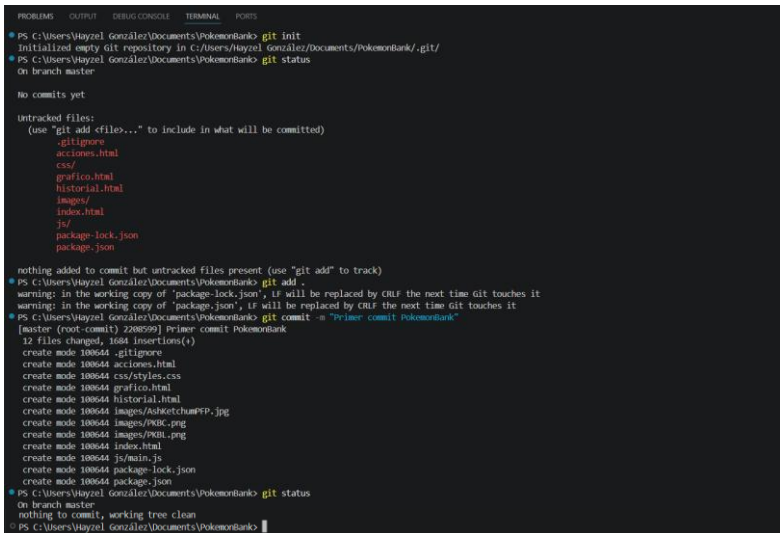
#	Tipo	Monto
1	Depósito	$25.00
2	Retiro	$10.00
3	Depósito	$24.00

Versionamiento del código

El versionamiento con Git fue importante en este proyecto porque ayudó a llevar un mejor control de los cambios que se hicieron en cada archivo. También permitió guardar avances de forma ordenada, identificar qué se modificó y mantener un historial del desarrollo de Pokémon Bank. Además, su uso demuestra buenas prácticas de programación y una mejor organización del trabajo.

Primer commit

En la primera terminal se realizó la creación del repositorio con ***git init***. Después, con ***git status***, se revisó el estado de los archivos, luego se usó ***git add .*** para agregarlos y finalmente se hizo el ***commit "Primer commit PokemonBank"***. Este paso sirvió para guardar la primera versión completa del proyecto.



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git init
Initialized empty Git repository in C:/Users/Hayzel González/Documents/PokemonBank/.git/
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        acciones.html
        css/
        grafico.html
        historial.html
        images/
        index.html
        js/
        package-lock.json
        package.json

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\Hayzel González\Documents\PokemonBank> git add .
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Primer commit PokemonBank"
[master (root-commit) 2208599] Primer commit PokemonBank
 12 files changed, 1684 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 acciones.html
 create mode 100644 css/styles.css
 create mode 100644 grafico.html
 create mode 100644 historial.html
 create mode 100644 images/AshKetchumPFP.jpg
 create mode 100644 images/PKDC.png
 create mode 100644 images/PKDL.png
 create mode 100644 index.html
 create mode 100644 js/main.js
 create mode 100644 package-lock.json
 create mode 100644 package.json
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

main.js

En esta terminal se guardaron los cambios hechos en el archivo ***main.js***. Primero se agregó el archivo con ***git add js/main.js*** y luego se registró con el ***commit "Actualizar main.js"***. Así quedó guardada una nueva versión de ese archivo.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   js/main.js

PS C:\Users\Hayzel González\Documents\PokemonBank> git add js/main.js
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar main.js"
[master 7b2f871] Actualizar main.js
 1 file changed, 1 insertion(+), 3 deletions(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
7b2f871 (HEAD -> master) Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

styles.css

Aquí se versionaron los cambios realizados en ***styles.css***. Git detectó que el archivo fue modificado, luego se agregó con ***git add css/styles.css*** y finalmente se guardó con el ***commit "Actualizar styles.css"***.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/styles.css

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add css/styles.css
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar styles.css"
[master 51c96d0] Actualizar styles.css
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
51c96d0 (HEAD -> master) Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

index.html

En este bloque se registraron los cambios hechos en ***index.html***. Después de revisar el estado del archivo, se agregó con ***git add index.html*** y se guardó con el ***commit "Actualizar index.html"***.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add index.html
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar index.html"
[master ab33188] Actualizar index.html
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
ab33188 (HEAD -> master) Actualizar index.html
51c96d0 Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

acciones.html

En esta terminal se guardaron los cambios de ***acciones.html***. Para eso, se revisó el archivo modificado, se agregó con ***git add acciones.html*** y luego se hizo el ***commit "Actualizar acciones.html"***.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   historial.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add historial.html
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar historial.html"
[master f7aace6] Actualizar historial.html
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
f7aace6 (HEAD -> master) Actualizar historial.html
1d3912f Actualizar acciones.html
ab33188 Actualizar index.html
51c96d0 Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

historial.html

Aquí se registraron los cambios realizados en ***historial.html***. El archivo fue agregado con ***git add historial.html*** y luego se guardó con el ***commit "Actualizar historial.html"***.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   historial.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add historial.html
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar historial.html"
[master f7aace6] Actualizar historial.html
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
f7aace6 (HEAD -> master) Actualizar historial.html
1d3912f Actualizar acciones.html
ab33188 Actualizar index.html
51c96d0 Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

grafico.html

Aquí se registraron los cambios realizados en ***historial.html***. El archivo fue agregado con ***git add historial.html*** y luego se guardó con el ***commit "Actualizar historial.html"***.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   grafico.html

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add grafico.html
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Actualizar grafico.html"
[master 76d49d3] Actualizar grafico.html
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
76d49d3 (HEAD -> master) Actualizar grafico.html
f7aace6 Actualizar historial.html
1d3912f Actualizar acciones.html
ab33188 Actualizar index.html
51c96d0 Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

Último commit

Finalmente, se hizo un ***commit*** general con el mensaje ***"Último commit PokemonBank"***. En este paso se agregaron varios cambios pendientes con ***git add .*** y se guardó la versión final del proyecto, dejando el repositorio limpio y actualizado.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   acciones.html
        modified:   css/styles.css
        modified:   grafico.html
        modified:   css/styles.css
        modified:   css/styles.css
        modified:   grafico.html
        modified:   historial.html
        modified:   index.html
        modified:   js/main.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Hayzel González\Documents\PokemonBank> git add .
PS C:\Users\Hayzel González\Documents\PokemonBank> git commit -m "Último commit PokemonBank"
[master d568dd4] Último commit PokemonBank
 6 files changed, 7 insertions(+), 6 deletions(-)
PS C:\Users\Hayzel González\Documents\PokemonBank> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\Hayzel González\Documents\PokemonBank> git log --oneline
d568dd4 (HEAD -> master) Último commit PokemonBank
76d49d3 Actualizar grafico.html
f7aace6 Actualizar historial.html
1d3912f Actualizar acciones.html
ab33188 Actualizar index.html
51c96d0 Actualizar styles.css
7b2f871 Actualizar main.js
2208599 Primer commit PokemonBank
PS C:\Users\Hayzel González\Documents\PokemonBank>
```

Conclusiones

El proyecto Pokémon Bank culmina como una implementación práctica y funcional que integra con éxito las tecnologías esenciales de HTML, CSS y JavaScript. A través de este desarrollo, se logró consolidar el dominio del frontend, transformando procesos complejos de un cajero automático real en una interfaz dinámica, intuitiva y profesional.

La solución no solo cumple con los requisitos técnicos de autenticación y gestión de transacciones, sino que también subraya la importancia del diseño centrado en el usuario y la visualización clara de datos. En definitiva, este proyecto representa un avance significativo en el fortalecimiento de buenas prácticas de programación y en la capacidad de construir aplicaciones web interactivas que responden con eficiencia a las demandas del entorno digital actual.